

Chapter 11: Trees

Summary & Important Topics

1. Introduction to Trees (Section 11.1)

Basic Definitions

- **Tree:** A connected undirected graph with no simple circuits.
- **Forest:** A graph that has no simple circuits but is not connected. Each connected component of a forest is a tree.
- **Theorem:** An undirected graph is a tree if and only if there is a **unique simple path** between any two of its vertices.

Rooted Trees

- **Rooted Tree:** A tree in which one vertex has been designated as the *root* and every edge is directed away from the root.
- **Terminology:**
 - **Parent/Child:** If $u \rightarrow v$ is a directed edge, u is the parent and v is the child.
 - **Siblings:** Vertices with the same parent.
 - **Ancestors:** Vertices on the path from the root to a vertex (excluding the vertex itself).
 - **Descendants:** Vertices that have v as an ancestor.
 - **Leaf:** A vertex with no children.
 - **Internal Vertex:** A vertex that has children.

m-ary Trees

- **m-ary Tree:** A rooted tree where every internal vertex has no more than m children.
- **Full m-ary Tree:** A rooted tree where every internal vertex has **exactly** m children.
- **Binary Tree:** An m-ary tree with $m = 2$.
- **Ordered Rooted Tree:** The children of each internal vertex are ordered (e.g., Left Child, Right Child).

Key Formulas & Properties (Paper Point of View)

These formulas are critical for solving calculation problems:

1. **Edges in a Tree:** A tree with n vertices has exactly $n - 1$ edges. 2. **Vertices in a Full m-ary Tree:**

- Let n be the total vertices.
- Let i be the internal vertices.
- Let l be the leaves.
- **Relation:** $n = mi + 1$
- **Leaves Relation:** $l = (m - 1)i + 1$
- **Internal vs Leaves:** $i = \frac{l-1}{m-1}$

3. Height of a Tree:

- **Level:** The length of the path from the root to the vertex (Root is level 0).
- **Height (h):** The maximum level of vertices.
- **Leaf Bound:** There are at most m^h leaves in an m-ary tree of height h .
- **Height Bound:** $h \geq \lceil \log_m l \rceil$

2. Tree Traversal (Section 11.3)

Traversals are methods for systematically visiting every vertex of an ordered rooted tree.

Traversal Methods

1. Preorder Traversal (Root → Left → Right):

- Step 1: Visit Root.
- Step 2: Traverse subtrees from left to right in preorder.
- *Application:* Prefix notation (Polish notation).

2. Inorder Traversal (Left → Root → Right):

- Step 1: Traverse the leftmost subtree in inorder.
- Step 2: Visit Root.
- Step 3: Traverse the remaining subtrees in inorder.
- *Application:* Infix notation (Standard arithmetic expressions).

3. Postorder Traversal (Left → Right → Root):

- Step 1: Traverse subtrees from left to right in postorder.
- Step 2: Visit Root.
- *Application:* Postfix notation (Reverse Polish notation).

3. Spanning Trees (Section 11.4)

Definitions

- **Spanning Tree:** A subgraph of a simple graph G that is a tree and contains **every vertex** of G .
- **Theorem:** A simple graph is connected if and only if it has a spanning tree.

Algorithms to Find Spanning Trees

You should know the difference in the order of vertex visitation for these two algorithms.

1. Depth-First Search (DFS):

- Strategy: Go "deep". Visit a child, then that child's child, etc., before backtracking.
- Uses **Backtracking**.
- Implementation often uses a **Stack** or Recursion.

2. Breadth-First Search (BFS):

- Strategy: Go "wide". Visit all neighbors of the current vertex (Level 1), then all neighbors of those neighbors (Level 2).
- Implementation uses a **Queue**.